

Examen ordinario UD1 + UD2 (2 horas)

Contexto de la prueba

Esta prueba ordinaria integra contenidos de:

- UD1: scripting en Linux con **bash**, argumentos, validaciones, condicionales, bucles y tratamiento de ficheros.
- UD2: trabajo en contenedores LXD/LXC, paqueteria, servicios, procesos y programacion de tareas.

La prueba se realiza sobre un unico contenedor Ubuntu. La Parte 1 se hace en el host. Las Partes 2 a 5 se hacen solo dentro del contenedor.

Material permitido: **man**, **--help**, apuntes de clase y el material de **docs/UD1** y **docs/UD2**.

Organizacion y entrega

Dentro del contenedor crea la carpeta `~/ordinario_ud1_ud2/` con esta estructura:

- `~/ordinario_ud1_ud2/scripts/`
- `~/ordinario_ud1_ud2/resultados/`
- `~/ordinario_ud1_ud2/datos/`
- `~/ordinario_ud1_ud2/cron/`

Guarda en esa carpeta todos los scripts, evidencias y ficheros pedidos.

Entrega final:

1. Genera en el contenedor el fichero `~/ordinario_ud1_ud2_entrega.tar.gz` con toda la carpeta de trabajo.
2. Descarga ese fichero al host.
3. Sube a Aules `ordinario_ud1_ud2_entrega.tar.gz`.

Parte 1 - Preparacion del laboratorio LXD/LXC (1 punto)

1. Crea un contenedor Ubuntu 24.04 llamado `ord-ud1ud2`.
2. Configura antes del primer arranque:
 - un limite de memoria de 512MB;
 - arranque automatico del contenedor.
3. Arranca el contenedor y comprueba su estado e IP.
4. Entra en el contenedor y crea la estructura de carpetas pedida.
5. Guarda en `~/ordinario_ud1_ud2/resultados/lxc.txt` evidencias de:
 - creacion y arranque del contenedor;
 - informacion general del contenedor;
 - configuracion de memoria;
 - configuracion de arranque automatico.
6. Crea un snapshot llamado `inicio`.

7. Realiza dentro del contenedor las Partes 2 a 5.

Parte 2 - Datos y scripting en bash (5 puntos)

1. Crea ~/ordinario_ud1_ud2/datos/inventario.txt con este contenido:

```
# ip nombre disco ram servicio estado
10.0.0.21 web01 12 4 nginx activo
10.0.0.22 app01 28 2 apache2 mantenimiento
10.0.0.23 db01 40 8 mariadb activo
10.0.0.24 cache01 xx 16 redis activo
10.0.0.25 bk01 9 1 rsync caido
10.0.0.26 api01 18 6 nginx activo
10.0.0.27 mon01 30 yy prometheus activo
```

2. Crea el script ~/ordinario_ud1_ud2/scripts/revision_inventario.sh.

3. El script debe cumplir todo lo siguiente:

- estar hecho en **bash** y tener permisos de ejecucion;
- incluir comentarios breves en las partes principales;
- recibir exactamente dos argumentos numericos: **umbral_disco** y **umbral_ram**;
- validar el numero de argumentos y que ambos sean enteros;
- validar que existe ~/ordinario_ud1_ud2/datos/inventario.txt;
- ignorar lineas vacias o que empiecen por #;
- validar que cada linea util tenga exactamente 6 campos;
- si una linea no tiene el formato correcto, mostrar: **ERROR FORMATO: linea <n>**;
- si disco o ram no son enteros, mostrar: **ERROR DATOS: <nombre> (<ip>)**;
- si disco es menor que **umbral_disco**, mostrar: **DISCO BAJO: <nombre> (<ip>) -> <disco>GB**;
- si ram es menor que **umbral_ram**, mostrar: **RAM BAJA: <nombre> (<ip>) -> <ram>GB**;
- si estado no es activo, mostrar: **SERVICIO NO OPERATIVO: <nombre> -> <estado>**;
- clasificar cada servicio en uno de estos grupos: **web**, **datos**, **copias** u **otros**;
- generar ~/ordinario_ud1_ud2/resultados/resumen_inventario.csv con cabecera y este formato: **ip,nombre,disco,ram,servicio,tipo,estado**;
- identificar el equipo valido con menor disco libre;
- al final mostrar: **Resumen: <n_validos> validos, <n_formato> formato, <n_datos> datos, <n_disco> disco, <n_ram> ram, <n_no_operativo> no_operativo**
- despues del resumen mostrar: **Minimo disco: <nombre> (<ip>) -> <disco>GB**

4. Ejecuta el script con umbrales 20 4 y guarda la salida en `~/ordinario_ud1_ud2/resultados/revision_2`.
5. Guarda una copia del contenido del script en `~/ordinario_ud1_ud2/resultados/revision_script.txt`.

Parte 3 - Paqueteria y servicios (2 puntos)

1. Actualiza el indice de paquetes y deja constancia de los paquetes actualizables.
2. Instala `nginx` y `curl`.
3. Personaliza `/var/www/html/index.html` para que aparezca:
 - el texto `Ordinario UD1 UD2`;
 - el hostname del contenedor.
4. Guarda en `~/ordinario_ud1_ud2/resultados/servicios.txt`:
 - evidencias de actualizacion e instalacion;
 - un listado de paquetes instalados relacionados con `ssh`, `nginx` y `curl`;
 - informacion de busqueda y dependencias de `nginx`;
 - informacion basica del paquete `nginx`;
 - un listado de ficheros relevantes del paquete `nginx`;
 - el estado del servicio `nginx`;
 - los ultimos 20 registros del servicio `nginx`;
 - la comprobacion de que `nginx` esta escuchando en red;
 - el objetivo por defecto del sistema;
 - el estado de las unidades de `nginx` y `cron`;
 - la comprobacion de acceso local a la web del contenedor.
5. Deshabilita el arranque automatico de `nginx`.
6. Deten y arranca manualmente `nginx` de nuevo.
7. Guarda tambien en el mismo fichero:
 - la evidencia del nuevo estado de habilitacion de `nginx`;
 - una nueva comprobacion del estado del servicio;
 - una nueva comprobacion de escucha en red.

Parte 4 - Procesos y tareas programadas (1 punto)

1. Lanza `sleep 600` y `sleep 900` en segundo plano.
2. Guarda en `~/ordinario_ud1_ud2/resultados/procesos.txt`:
 - los PID de ambos procesos;
 - la relacion de trabajos en segundo plano;
 - una lista de procesos donde aparezcan ambos `sleep`;
 - la relacion de senales disponibles;
 - la evidencia del cambio de niceness del `sleep 600` a +5;
 - la comprobacion de la nueva prioridad del `sleep 600`;
 - la comprobacion de que `sleep 900` ha terminado tras enviar `SIGTERM`;
 - la comprobacion de que `sleep 600` ha terminado tras enviar `SIGKILL`.
3. Crea una tarea programada del usuario actual que cada 15 minutos anada la fecha a `~/ordinario_ud1_ud2/cron/fechas.log`.
4. Anade una segunda tarea programada que al minuto 5 de cada hora anada

- uptime a ~/ordinario_ud1_ud2/cron/carga.log.
5. Guarda el contenido final de la programacion en ~/ordinario_ud1_ud2/cron/crontab.txt.

Parte 5 - Systemd timer y empaquetado final (1 punto)

1. Crea /etc/systemd/system/estado.service para que anada date y uptime a ~/ordinario_ud1_ud2/cron/estado.log.
2. Crea /etc/systemd/system/estado.timer para ejecutar ese servicio cada 30 minutos usando OnCalendar y Persistent=true.
3. Recarga la configuracion de systemd.
4. Habilita y arranca el timer.
5. Ejecuta manualmente una vez estado.service.
6. Copia estado.service y estado.timer a ~/ordinario_ud1_ud2/cron/.
7. Guarda en ~/ordinario_ud1_ud2/resultados/timer.txt:
 - el estado actual del timer;
 - la evidencia de su siguiente ejecucion programada;
 - los registros recientes del servicio asociado.
8. Genera ~/ordinario_ud1_ud2_entrega.tar.gz y preparalo para su descarga desde el host.

Puntuacion detallada (sobre 10)

Parte 1 - 1 punto

- 0,25: creacion correcta del contenedor y arranque posterior.
- 0,25: configuracion correcta de memoria y autostart.
- 0,25: estructura de carpetas y evidencias guardadas en lxc.txt.
- 0,25: snapshot inicio creado correctamente.

Parte 2 - 5 puntos

- 0,50: fichero inventario.txt creado correctamente.
- 0,75: validacion de argumentos y existencia del fichero.
- 0,75: lectura correcta del fichero ignorando comentarios y vacios.
- 0,75: deteccion de errores de formato y de datos.
- 0,75: alertas correctas de disco, RAM y servicio no operativo.
- 0,50: uso correcto de estructuras de control y validaciones.
- 0,50: generacion correcta de resumen_inventario.csv.
- 0,50: calculo del minimo de disco y resumen final.

Parte 3 - 2 puntos

- 0,50: actualizacion e instalacion de paquetes.
- 0,50: personalizacion funcional de index.html.
- 0,50: evidencias completas de paqueteria y consulta de informacion.
- 0,50: comprobacion, logs, red y gestion manual de nginx.

Parte 4 - 1 punto

- 0,50: gestion correcta de procesos, prioridades y senales.
- 0,50: tareas programadas correctas y `crontab.txt` guardado.

Parte 5 - 1 punto

- 0,50: `service` y `timer` correctos, habilitados y arrancados.
- 0,50: evidencias del timer, logs y empaquetado final correcto.

Criterios generales de correccion

- La estructura de entrega debe coincidir con la pedida.
- Los ficheros deben estar en las rutas indicadas. Si no, **no se corregiran**.
- Se valorara la claridad del script, el uso correcto de permisos y la organizacion de las evidencias.
- Los comandos deben ser funcionales dentro del contenedor y coherentes con el trabajo realizado.